RETAIL DATA AGENT · BRAND SCOPING · AS OF 2026-10-31

How does a brand manager see only their own brands?

The AI writes SQL as if it could see every brand. Before that SQL runs, our code swaps every table in it for a smaller table that holds only the user's brands. The AI can ask for anything, but other brands' rows are simply not there. See also one query's full journey, the architecture and what happens when a query fails.

🟧 What the AI wrote 🟩 Filtered to the user's brands 🟦 BigQuery 🟥 Stopped

In short

1 The user is picked at startup (

`--`), and
`user`
`levis_manager`

`users.yaml` says which brands they see. The CEO has

`all` .

2 The filter is applied by code to every query, not checked for and not asked of the AI, so no prompt can talk its way around it.

3 Only

`products` has a brand. The other tables are filtered through the IDs that link them to those products.

Where the filter sits

1 · THE AI WRITES SQL
Plain SQL over the four real tables. It doesn't need to filter by brand, and no filter it writes can widen what it sees.

2 · GUARD AND PERSONAL DATA
One SELECT, only the four allowed tables, only the columns in `schema.yaml`, no `*`, nothing it can't parse. Then customer ids become pseudonyms such as `cust_72ac84091163`, for every user. See the parser rules.
× anything else is stopped here

3 · BRAND SCOPE (NEW)
Swaps each table for its brand-filtered version, including the real tables inside the pseudonym copies, so a manager's pseudonyms come from their own customers only. The CEO skips this step.

4 · SIZE CHECK
A free dry run of the *filtered* SQL: over 1 GB is stopped.

5 · BIGQUERY RUNS IT
The rows that come back belong to the user's brands only.

How each table reaches a brand

Only the product catalog knows brands. Everything else follows the links back to it, the way a receipt leads back to the shop that sold the item.

`products`
`WHERE brand IN ('Levi\'s')`
Only the user's brands. This is the root of every other filter.

`order_items`
`product_id IN (those products)`
Items sold of those products.

`orders`
`order_id IN (those items' orders)`
Orders with at least one such item.

`users`
`id IN (those items' buyers)`
Customers who bought at least one.

A real example: `levis_manager` counts products

1 The AI writes the obvious query, with no brand filter at all:

```
SELECT brand, COUNT(*) AS n
FROM `bigquery-public-data.thelook_ecommerce.products`
GROUP BY brand
```

2 The brand scope adds the four filtered tables at the front and points the query at them. The AI's own part is unchanged:

```
WITH _brand_scoped_products AS (
  SELECT * FROM `bigquery-public-data.thelook_ecommerce.products` AS products
  WHERE brand IN ('Levi\'s')),           -- built by our code, from users.yaml
  _brand_scoped_order_items AS (…),       -- the other three are always added too;
  _brand_scoped_orders AS (…),            -- BigQuery skips the ones a query
  _brand_scoped_users AS (…)              -- doesn't read, at no extra cost
SELECT brand, COUNT(*) AS n
FROM _brand_scoped_products AS products  -- old name kept as alias
GROUP BY brand
```

3 BigQuery returns one row: `Levi's · 259`. For `denim_lead` (Levi's and Calvin Klein) the same query returns two rows, `Calvin Klein · 497` and `Levi's · 259`. That was checked against the live data on 2026-10-31, and it scanned 0.35 MB.

Two ways to do it, and why we rewrite the SQL

A · REWRITE THE SQL · CHOSEN

Our code edits each query before it runs, using the same SQL parser the guard already uses.

- + Nothing to set up: works with any reviewer's key and project.
- + A new user is one line in `users.yaml`
- + Fully tested offline, with tricky queries.
- – The rule lives in our code, so only queries that go through the gateway are protected.

B · SCOPED VIEWS · PUT ASIDE

Pre-filtered views per user, stored in BigQuery. The gateway only points the query at that user's views.

- + The filter lives in the database, so other tools get it too.
- + In production, each login could read only its own views.
- – Needs datasets and views created in an external service (a GCP project), and kept in step with `users.yaml`
- – Can't be tested without live BigQuery, and the gateway still has to swap table names.

For a prototype that reviewers run with their own free key, the setup cost of B decides it. B remains the natural production upgrade: the same four filters, moved into views behind per-user access.

Tricky queries it has to survive

THE AI WRITES...	WHAT HAPPENS	
`SELECT brand, name FROM products`, no filter	Reads the filtered table, so only the user's brands come back. (A `SELECT *` is stopped earlier by the personal-data check; only our own filtered tables use `*`.)	**covered**
`WHERE brand = 'Nike'` on another brand	Filters an already-filtered table, so it returns nothing.	**covered**
Its own `WITH products AS (…)`	Its WITH reads our filtered table, because ours come first.	**covered**
Subqueries, `UNION`s, a table used twice	Every table reference in the parsed query is swapped.	**covered**
A WITH named like ours, `_brand_scoped_…`	BigQuery rejects the duplicate name, so the query fails rather than leaks.	**fails closed**
A table with no brand rule	Stopped: "can't be limited to your brands".	**fails closed**
`orders.num_of_item`	Recounted to the user's own items, so it no longer counts other brands' items. See why.	**fixed**

> **The rule in one sentence**
> Never ask the AI to respect brands: swap every table it reads for one that already holds only the user's brands, and stop any query where that can't be done.

Code: `BrandScope.apply` in `src/retail_agent/query_safety/step3_brand_scope/brand_scope.py` (the four filters are in `_FILTERS`), called by `QueryGateway.run` in `src/retail_agent/query_safety/gateway.py` after `QueryGuard.check`, `PersonalDataCheck.check` (`query_safety/step2_personal_data/columns.py`) and `CustomerPseudonyms.apply` (`query_safety/step2_personal_data/customer_pseudonyms.py`), and before the byte check. Built in `assemble` in `src/retail_agent/app.py` from `schema.dataset` and `user.brands`. The user comes from `load_user` in `src/retail_agent/users.py`, reading `users.yaml` (`USERS_PATH`). The filtered SQL is logged as `Rewritten SQL:` in `logs/retail-agent.log`.

Tests: `tests/test_step3_brand_scope.py` (the tricky queries above, the CEO, the gateway end to end), `tests/test_step2_customer_pseudonyms.py` (both rewrites together) and `tests/test_users.py`.

Plan: step 3 in `docs/plan.md`. See also one query's full journey, every rule that uses the parse tree, why num_of_item is recounted, architecture, SQL errors.